Lie sqrt-residual vs. metric NTFields baseline: T-shape env1 evaluation

Oct 7, 2026 · @jeffrey

Summary

With the forward planner, the (sqrt(N)-1)^2 Lie field clearly beats bare metric NTFields. With the SD planner the two are tied on success, but the Lie two-route network collides half as often. All figures are averages over 11 T-shape env1 test sets; the 4-pair nonlinear set is excluded.

Average over 11 sets	Lie sqrt, original	Lie sqrt, two-route	Metric baseline, mean of 3 seeds (range)
Forward success %	63.1	**66.3**	55.7 (53.7–58.7)
Forward convergence %	78.4	**83.1**	68.8 (66.0–72.1)
Forward collision %	28.7	**24.0**	29.6 (28.8–30.3)
SD success %	76.8	78.8	77.4 (75.5–**81.3**)
SD convergence %	81.6	82.8	82.3 (79.6–**86.3**)
SD collision %	12.3	**5.9**	11.8 (10.2–13.8)
Path-caused grazing, forward, % of successes	**11.1**	18.7	12.9 (11.5–14.0)
Path-caused grazing, SD, % of successes	4.1	**3.1**	3.5 (3.3–3.8)
Median penetration of colliding paths, forward (x1e-3)	13.4	**5.6**	13.1

- **Forward planner.** Lie original beats all three baseline seeds on all 11 sets; two-route does so on 8. The gap is widest on long or high-rotation queries: exp2/exp3 (+14 to +23 points over the baseline mean) and bigangle (two-route 57.6% vs 27.7–34.6%). Most of the gain is convergence (+10 to +14 points), with no rise in collisions. In case-by-case pairs, two-route solves 3135 cases the baseline misses and misses 1986 it solves.

- **SD planner.** On average neither model wins: baseline seed 3 (81.3%) beats both Lie models, while seeds 1 and 2 (75.5%) lose to both. Two-route still wins the hardest sets: bigangle 74.9% vs 48.2–66.5%, exp3 70.5% vs 55.0–67.5%. It loses on transonly (72.0% vs 80.8–85.5%) and 1k (85.2% vs 89.5–91.3%).
- **Collisions and grazing.** Two-route trades grazing for depth. With the forward planner it skims obstacles more often (18.7% vs about 12% of successes), but its collisions are about 2.3× shallower. With SD it has the lowest collision rate on every set except transonly, with grazing at baseline level.
- **Seed noise matters.** The three baseline seeds differ by up to 12.5 points on a single set (exp3, SD). There is one Lie seed per architecture, so per-set SD differences under about 10 points are not significant.

Setup

Five frozen checkpoints, all trained on `datasets/3dshape/Tshape3d_env1` for 4000 epochs. Nothing was retrained. Each one runs on 12 test sets with two planners: 120 runs in total.

Model	Package	Loss	Network	Checkpoint
Lie sqrt, original	`lie_metric`	anisotropic, (sqrt(N)-1)^2	original	ablation A1, ValLoss 1.500e-3
Lie sqrt, two-route	`lie_metric`	anisotropic, (sqrt(N)-1)^2	two_route	ablation A3, ValLoss 1.497e-3
Metric baseline, seed 1	`metric_baseline`	isotropic, (sqrt(S|grad tau|)-1)^2	original (only option)	`3dshape_metric_baseline/...ValLoss_1.808e-03`
Metric baseline, seed 2	`metric_baseline`	same	original	`compare/Tshape3d_env1_metric_s2`
Metric baseline, seed 3	`metric_baseline`	same	original	`compare/Tshape3d_env1_metric_s3`

We have three baseline seeds but only one Lie seed per architecture. The baseline's seed-to-seed spread therefore stands in for run-to-run noise.

Test sets. These are all the distinct T-shape env1 sets in `ntrlshape` and this repo; sets with identical `sampled_points.npy` hashes were merged. `testing_data_old_dontuse` was skipped.

Set	Pairs	Endpoint offset	What it tests
1k	1000	0.02	Standard set. Every pair has a verified RRT-Connect path
tight (old)	1000	0.005	Older tight set (`ntrlshape/.../testing_data/Tshape3d_env1_tight`)
tight_3d	1000	0.005	Tight-clearance population
tight001	1000	0.001	Tightest clearance
exp1 non-linear	200	0.005	tight_3d pairs whose straight line collides
exp2 farthest	200	0.005	tight_3d pairs farthest apart in position
exp3 hard+far	200	0.005	Non-linear pairs ranked by distance
exp4 farthest-500	500	0.005	Same rule as exp2, top 500
rotheavy	200	0.005	Translation at most 0.05, rotation at least 90 deg
bigangle	200	0.005	Both endpoint rotations in [135, 180) deg
transonly	200	0.005	Same rotation, translation at least 0.3 (control)
nonlinear (4)	4	0.02	`nonlinear_testing`: straight line collides. Too small to rank models

Planners. Both planners use 50 samples, horizon 5, step 0.015, temperature 50, 200 iterations, and a convergence ball of 0.01.

- *Forward*: one-directional MPPI on tau, copied from `evaluate_training_3d_batched.MPPI_batched` at its defaults.
- *SD*: `spread_planner.py` with `--steer-bias 0.5 --no-gate`, as in the README. It rolls out from both ends in turn, adds a local slowness term (weight 0.03), and steers along the fitted local speed gradient.

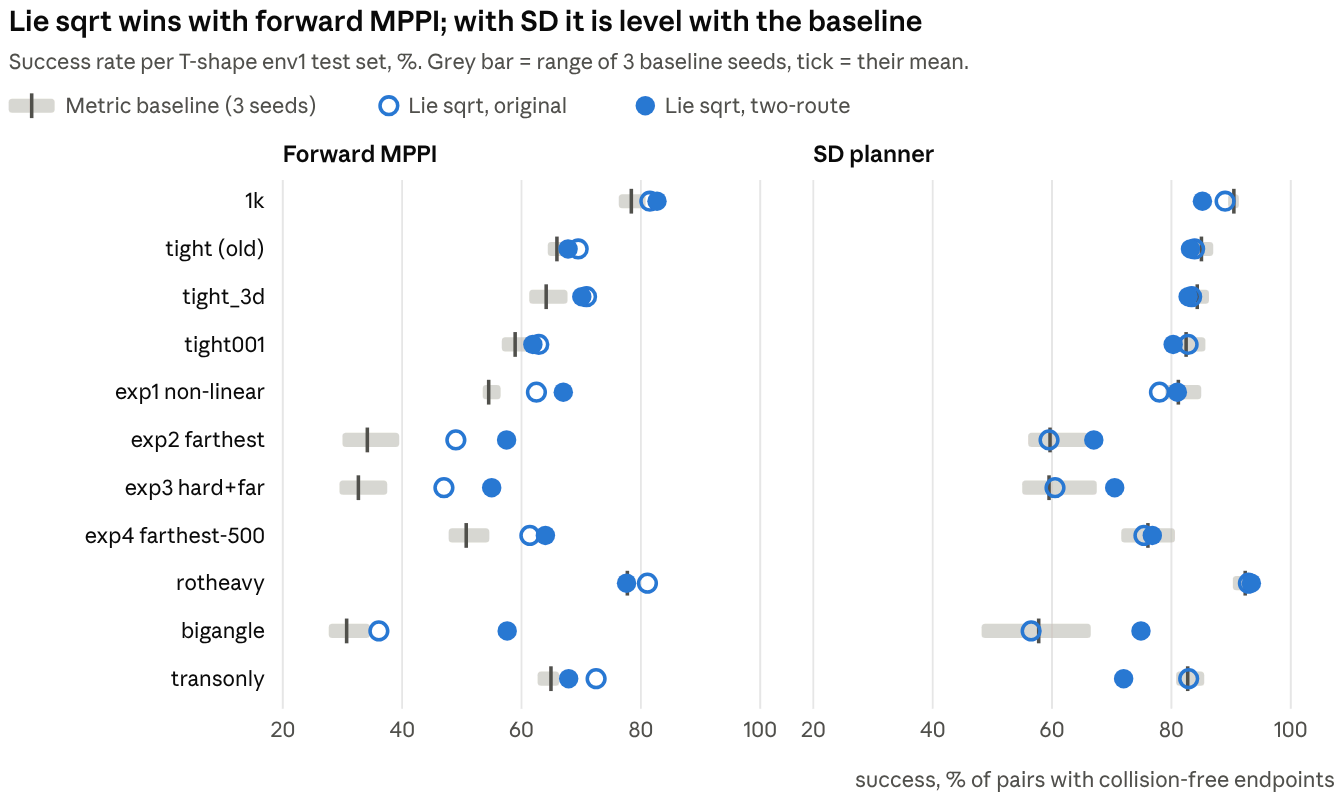
Metrics (per test set, over pairs whose endpoints are collision-free):

- *Convergence*: the planner reached the goal ball.

- *Collision*: any waypoint has an environment point inside the placed shape. Collisions are checked against 50k env surface points (seed 0), the evaluator's own test.
- *Success*: converged and no collision.
- *Grazing*: signed clearance (capped at 0.05) at every waypoint and at 3 interpolants per segment. Grazing = a successful path whose minimum clearance falls below 0.005, which is the tight sets' endpoint offset. A *hidden collision* = a successful path that penetrates only between waypoints. Clearances are in normalized units, where 1 = the environment's largest extent.

The harness reproduces last session's numbers exactly: Lie sqrt (original) with the forward planner on tight (old) gives 963 cases, 69.5% success, 23.4% collision.

Results



120 runs of eval_suite.py on 2026-10-07 · 11 test sets × 5 checkpoints × 2 planners (nonlinear, 4 pairs, omitted)

With forward MPPI, the blue markers sit right of the grey baseline band on almost every row. With SD they fall inside it, except two-route on exp2, exp3 and bigangle (ahead) and on transonly and 1k (behind).

Forward MPPI. Each cell reads Lie original · Lie two-route · baseline mean (min–max over 3 seeds). Grazing counts successful paths whose clearance (waypoints plus 3 interpolants per segment) drops below 0.005 while both endpoints are at or above 0.005.

Set	n	Converged %	Collision %	Path-caused grazing, % of successes	Median penetration, x1e-3
1k	1000	88.3 · 91.4 · 83.0 (82.0–85.0)	11.6 · 10.4 · 12.1 (11.3–13.2)	5.8 · 11.1 · 5.1 (4.7–5.4)	11.6 · 4.7 · 14.7 (11.7–18.7)
tight (old)	963	83.2 · 84.0 · 77.9 (76.3–80.7)	23.4 · 23.1 · 23.6 (22.4–24.4)	11.7 · 15.8 · 11.1 (10.9–11.5)	11.5 · 4.4 · 11.1 (10.1–11.7)
tight_3d	962	85.0 · 85.7 · 77.2 (74.6–79.9)	23.2 · 21.9 · 25.3 (23.1–26.7)	11.6 · 18.7 · 12.2 (11.4–13.1)	11.3 · 5.7 · 10.8 (9.9–12.3)
tight001	856	82.8 · 83.9 · 75.9 (73.7–78.4)	29.9 · 30.3 · 29.9 (28.4–31.5)	9.5 · 16.0 · 10.3 (9.7–10.7)	10.1 · 5.7 · 10.3 (9.6–11.7)
exp1 non-linear	200	80.0 · 86.5 · 71.5 (66.5–75.5)	29.5 · 26.0 · 33.0 (29.0–37.5)	8.8 · 14.9 · 10.2 (5.3–16.8)	14.0 · 6.0 · 11.0 (8.7–14.1)
exp2 farthest	200	61.5 · 73.5 · 42.7 (36.5–48.5)	39.0 · 28.0 · 37.2 (35.5–40.0)	14.3 · 25.2 · 12.7 (6.3–16.7)	16.9 · 6.3 · 18.9 (16.1–21.9)
exp3 hard+far	200	61.0 · 72.5 · 42.3 (39.0–47.5)	39.5 · 30.0 · 39.3 (36.0–41.5)	11.7 · 25.5 · 14.1 (9.3–16.9)	15.2 · 5.9 · 17.2 (15.1–20.3)
exp4 farthest-500	500	74.6 · 81.2 · 64.0 (58.8–68.0)	28.4 · 25.2 · 32.3 (30.8–33.2)	14.3 · 19.4 · 11.0 (10.4–11.7)	16.1 · 5.5 · 13.3 (11.0–15.1)
rotheavy	196	95.4 · 95.9 · 93.7 (91.8–94.9)	16.8 · 19.9 · 19.2 (18.4–20.4)	11.3 · 19.7 · 13.1 (11.3–15.0)	10.8 · 5.8 · 8.2 (5.8–10.5)
bigangle	191	61.8 · 80.6 · 48.7 (45.0–55.5)	51.3 · 33.0 · 49.6 (46.6–52.4)	10.1 · 26.4 · 28.0 (24.2–31.6)	20.0 · 5.3 · 17.7 (17.5–18.0)
transonly	193	89.1 · 79.3 · 79.3 (78.8–79.8)	23.3 · 16.1 · 24.2 (22.3–26.4)	13.6 · 13.0 · 14.1 (10.9–15.7)	9.6 · 6.3 · 10.6 (7.8–12.4)

SD planner. Same layout.

Set	n	Converged %	Collision %	Path-caused grazing, % of successes	Median penetration, x1e-3
1k	1000	90.5 · 86.8 · 91.5 (90.2–92.3)	4.6 · 2.2 · 3.8 (3.3–4.4)	1.1 · 1.5 · 0.6 (0.3–1.0)	14.1 · 4.7 · 12.7 (8.6–16.5)
tight (old)	963	87.4 · 86.7 · 89.0 (87.4–91.1)	8.4 · 5.2 · 8.4 (7.7–9.6)	3.7 · 4.0 · 3.3 (3.1–3.5)	14.5 · 5.2 · 9.9 (5.5–15.3)

Set	n	Converged %	Collision %	Path-caused grazing, % of successes	Median penetration, x1e-3
tight_3d	962	87.1 · 85.4 · 88.6 (86.9–90.6)	8.6 · 3.8 · 8.5 (8.0–9.1)	3.7 · 3.4 · 3.5 (2.4–4.1)	10.6 · 7.3 · 8.8 (6.4–12.1)
tight001	856	88.6 · 85.4 · 88.4 (85.9–91.6)	9.9 · 7.6 · 10.7 (8.6–12.4)	2.4 · 2.5 · 2.5 (2.0–2.7)	9.4 · 8.3 · 8.9 (6.2–10.8)
exp1 non-linear	200	84.5 · 84.5 · 86.8 (84.5–90.5)	15.5 · 4.5 · 11.2 (9.5–13.0)	4.5 · 2.5 · 5.0 (2.9–6.3)	15.9 · 3.3 · 10.5 (9.4–11.0)
exp2 farthest	200	63.5 · 72.5 · 64.5 (59.0–72.0)	19.0 · 8.5 · 18.5 (16.0–21.0)	3.4 · 3.7 · 3.1 (2.6–3.6)	12.4 · 2.6 · 11.4 (8.8–15.5)
exp3 hard+far	200	63.5 · 75.0 · 63.3 (56.5–72.5)	16.5 · 7.0 · 18.0 (14.5–21.5)	5.0 · 5.0 · 3.9 (3.6–4.4)	13.0 · 4.0 · 12.8 (10.6–14.0)
exp4 farthest-500	500	79.8 · 80.8 · 80.5 (75.8–85.4)	11.8 · 5.2 · 11.3 (10.2–12.8)	5.6 · 3.1 · 3.7 (3.6–3.7)	12.0 · 5.7 · 10.0 (9.2–11.0)
rotheavy	196	98.0 · 96.4 · 98.5 (97.4–99.0)	6.6 · 4.1 · 7.3 (5.6–9.7)	3.8 · 3.3 · 3.3 (2.2–4.0)	8.2 · 12.4 · 6.8 (2.1–13.9)
bigangle	191	64.9 · 79.1 · 64.6 (59.2–71.7)	23.6 · 9.4 · 22.9 (18.8–29.8)	6.5 · 4.9 · 7.5 (5.4–9.4)	17.3 · 11.3 · 15.3 (13.5–17.0)
transonly	193	90.2 · 78.2 · 89.6 (87.0–92.2)	10.4 · 7.8 · 8.8 (7.3–9.8)	5.6 · 0.7 · 2.1 (1.3–3.2)	12.3 · 17.0 · 8.3 (5.5–11.8)

nonlinear (4 pairs). Forward successes: original 3, two-route 4, baseline 2/3/3. SD successes: original 4, two-route 3, baseline 4/3/4. Four pairs are too few to separate the models.

Analysis

The Lie field gives a better descent direction, and the forward planner depends on it alone. Forward MPPI just follows tau toward the goal, so a better-shaped field shows up directly as convergence. Lie two-route converges on 72.5–80.6% of exp2/exp3/bigangle pairs versus 42–49% for the baseline mean. Its collision rate is no higher (28–33% vs 37–50%). The anisotropic loss gives rotation its own speed term, and these sets need long or large-rotation paths, so it pays off most there. On short, translation-dominated queries (1k, tight sets, rotheavy), the networks are within a few points of each other.

SD absorbs most of the field-quality gap. SD plans from both ends and adds a local-slowness term plus a steer along the sampled speed gradient. That raises every model by 12–23 points on average and squeezes them into one band. What is left is a split by query type:

- On large-rotation or long-haul queries, two-route stays ahead: bigangle +17.1, exp3 +11.0 and exp2 +7.3 points over the baseline mean.
- On transonly (pure translation) and 1k, it falls behind by 10.7 and 5.3 points. Transonly convergence drops to 78.2% vs 89.6%. The two-route network seems to waste effort on rotation when no rotation is needed.
- Lie original stays inside, or within 1 point of, the baseline seed range on every set.

Grazing vs collision is an architecture effect, not a loss effect. Lie original and the baseline have nearly the same clearance profile: path-caused grazing 11–14% (forward) and 3–4% (SD), median penetration 12.5–14 x1e-3. Two-route is different under the forward planner: it grazes about 1.5× as often (18.7%), but when it collides it penetrates only 5.6 x1e-3 on median, against about 13 x1e-3 for the others. So the two-route field keeps paths close to surfaces but rarely lets them cut deep. Under SD that closeness disappears (grazing 3.1%) and its collision rate halves (5.9% vs 11.8%).

Collisions between waypoints are rare among successes. Only 0.4–2.3% of successful paths penetrate between waypoints, and success barely changes when interpolants are counted (macro forward success drops 0.6–1.4 points, SD 0.3–0.7). The waypoint-only evaluator is therefore a fair success metric here. Two-route SD shows a large gap between waypoint collisions (5.9%) and interpolated collisions (19.2%), but that gap sits almost entirely in non-converged paths, which already count as failures.

Bottom line. For forward planning, the (sqrt(N)-1)^2 Lie field is a real improvement over metric NTFields: +7.4 (original) and +10.6 (two-route) points of success over the baseline mean, the most on hard sets. With the SD planner, use two-route if the workload has large rotations or long hauls and fewer collisions matter; on short or translation-only queries the baseline is as good or better. One Lie seed per architecture is not enough to call the SD averages; 2 more Lie seeds would settle it.

Caveats and reproduction

- **Seeds.** There is one Lie checkpoint per architecture against three baseline seeds. The baseline's seed spread reaches 12.5 points on a 200-pair set. On a 1000-pair set the binomial standard error is about 1.3 points; on a 200-pair set it is about 3.5.
- **Training budget.** All five checkpoints have 4000 epochs on the same dataset. The Lie runs were the ablation checkpoints (A1, A3) from the earlier session, trained with a monkeypatched loss. The network code matches the current `models/lie_metric`; only the loss differs.

- **Clearance is sampled.** Clearance and collision are measured against a 50k-point env surface cloud, not a mesh-mesh test. Its point spacing is about the size of the 0.005 grazing threshold, so grazing numbers are approximate. The same cloud (seed 0) was used for every run, so comparisons between models are paired.
- **Endpoint filter.** Pairs whose endpoints collide on this cloud are dropped: 37/1000 on tight (old) and tight_3d, 144/1000 on tight001, and 4–9 of 200 on rotheavy/bigangle/transonly. Every model sees the same pairs.
- **Raw vs path-caused grazing.** On the tight sets, 10–13% of successful pairs start or end below 0.005. The raw minimum-clearance rate (in `summary.json` as `graze`) counts those; the tables count only paths that dip below 0.005 when both endpoints don't.
- **Five runs were re-done.** They hit CUDA out-of-memory on the shared GPU 0 and were rerun unchanged on GPU 2 with the same seed.

Files. Everything is in `results/sqrt_vs_baseline_tshape_env1/`: `eval_suite.py` (the planners and metrics), `aggregate.py`, `summary.json`, `runs/<set>__<model>__<planner>.json` (per-case records), the 12 test sets, and copies of the two Lie checkpoints.

```
# inside the pytorchserver image, repo at /workspace, that folder at /scratch
cd /workspace
python /scratch/eval_suite.py lie_metric /scratch/ckpt/lie_sqrt_2route.pt
/scratch/testsets/exp3 sd /scratch/runs/exp3__lie_sqrt_2route__sd.json cuda:0
bash /scratch/run_all.sh        # the full 120-run sweep (skips finished
outputs)
python /scratch/aggregate.py   # tables + summary.json
```